

Autonomous Form Preparation for CE Priority DEV

Repo document for the agent implementing unattended Form Prep on Clarkson Evans Priority ~24.1 (bin.95).
Web: <https://prioritydev.clarksonevans.co.uk/> · SQL: 10.220.0.5\DEV (system + base)
Host: CE-PRIORITY-DEV1 · Audience: Eshbel / Jester / any unattended Windows agent
Supersedes and expands: CE_Priority_Autonomous_Form_Prep_Design.pdf (hybrid F).

Hard rule: never report a form prepared unless EXECPREPLOCK.UPD='N' AND LASTPREPDATE moved. Never leave the unprepared estate marked prepared. Never embed passwords in this repo.

1. Purpose and problem

Form changes (triggers, FORMCLMNSA expressions, indexes, FORMKEYS, FORMJOINS) do not take effect until Priority Form Preparation compiles them. On CE DEV the human path is: web UI → Form Preparation → Unprepared Forms. That blocks overnight agent loops (UAT unpark, trigger fixes, index repairs). CLI WINACTIV -P FORMPREP from non-interactive sessions often exits without flipping EXECPREPLOCK. A full-queue web run compiles a low-id subset and skips high-id custom forms (e.g. 101883+), which is exactly the ZCLA_* work.

The runner must (a) prepare a named set of 1–10 forms without a human at the console, (b) return real compile errors, (c) restore every parked row, (d) refuse to run against live/PRI.

2. Goals, non-goals, environment freeze

Goals

- Prepare one or more named EXEC forms unattended on DEV.
- Return a versioned JSON result: Prepared[], StillUnprepared[], Errors[], Dialogs[], park counts, duration, Ok.
- Finish a 1–10 form target set in minutes, not hours.
- Be safe against the 1000+ unprepared-form queue.
- Be callable from Eshbel/Jester as Prepare-Forms.ps1 with exit 0 only on Ok.

Non-goals

- Do not UPDATE EXECPREPLOCK SET UPD='N' and call that prepared. That skips compilation. Runtime then fails oddly.
- Do not prepare the entire unprepared estate in MVP unless -Scope AllUnprepared is explicitly passed AND a human change ticket is recorded.
- Do not invent a Priority REST/OData/Tabula prep API. Dictionary dumps are out of scope.
- Do not store passwords, connection strings with passwords, or Si session cookies in git.
- Do not auto-click Ignore Duplicate Values / index rebuild dialogs.
- Do not run against PRI / live. Refuse if SQL host or web host is not the DEV constants below.

Frozen environment constants (fail closed if mismatched)

Key	Allowed value
WebHost	prioritydev.clarksonevans.co.uk
SqlInstance	10.220.0.5\DEV
SqlAuth	Windows integrated (no password in repo)
PriorityRoot	C:\Priority
Bin	C:\Priority\bin.95
SystemPrep	C:\Priority\system\prep
DefaultCompany	base (confirm at implement time; log actual)
AgentWork	C:\Priority\tml\agent-formprep\
MutexName	Global\CE-DEV-FORMPREP
AllowedComputer	CE-PRIORITY-DEV1 (or current DEV RDP host — pin in config)

3. Decision: Hybrid F, with parked-CLI probe

Primary mechanism remains Hybrid F from the design note: isolate the named set by parking every other UPD='Y' row, run Form Prep against the tiny remaining queue, verify EXECPREPLOCK, scrape every error channel, restore the park. That is the only path that has produced UPD='N' + advanced LASTPREPDATE for high-id ZCLA forms on this instance.

Amendment vs the design note: after a successful park, try CLI FORMPREP (no form name — i.e. all currently unprepared) for a short bounded window. A 1–3 row queue may be why interactive CLI works and agent CLI on the full estate does not. If LASTPREPDATE does not move within CliTimeoutSeconds (default 60), fall through to web Form Prep. Never run CLI and web prep concurrently. Phase-2 “CLI first on the full queue” is still rejected.

Option	Use in this plan
A SQL flip UPD=N only	FORBIDDEN as a success path. Used only as the temporary park isolation, always restored.
B WINRUN WINACTIV -P FORMPREP	Secondary, only after park, 60s probe, then fall through.
C formprep.exe	Same class as B. Do not add a third launcher.
D Thick-client Form Generator	Fallback if web automation regresses. Not MVP.
E Web Form Prep full queue	Control experiment only (AT5). Not MVP success path.
F Hybrid park + prep + SQL + scrape	PRIMARY.
G Undocumented REST prep	Out of scope.

4. Repo layout to create

```

form-prep/
  README.md
  CE_Priority_Autonomous_Form_Prep_Build_Plan.pdf  (this file)
  config/dev.psd1
  src/Prepare-Forms.ps1                          (public CLI)
  src/CE.FormPrep.psm1                          (module)
  src/CE.FormPrep.psd1
  src/Private/Assert-Environment.ps1
  src/Private/Get-FormExec.ps1
  src/Private/Lock-FormPrepMutex.ps1
  src/Private/New-ParkSnapshot.ps1
  src/Private/Restore-ParkSnapshot.ps1
  src/Private/Invoke-CliFormPrep.ps1
  src/Private/Invoke-WebFormPrep.ps1
  src/Private/Get-PrepErrors.ps1
  src/Private/Assert-PostHooks.ps1
  sql/001_agent_formprep_park.sql
  sql/002_resolve_forms.sql
  sql/003_verify_execpreplock.sql
  hooks/formkeys.psd1                          (known repairs)
  hooks/Apply-FormKeysRepair.ps1
  schemas/prepare-forms-result.schema.json
  tests/AT1-already-prepared.ps1
  tests/AT2-broken-trigger.ps1
  tests/AT3-index-dialog.ps1
  tests/AT4-abort-restores.ps1
  tests/AT6-refuse-non-dev.ps1
  tests/AT7-auth-expired-no-park.ps1
  logs/                                         (gitignored)

```

5. Public interface

5.1 PowerShell

```

Import-Module .\src\CE.FormPrep.psd1

Prepare-Forms -Names @(
  'ZCLA_PARTLONGDESC',
  'ZCLA_PARTLONGDHIST',
  'ZCLA_PARTLONGDREV'
) `
-Environment DEV `
-TimeoutMinutes 15 `
-CliTimeoutSeconds 60 `
-RestoreQueue `

```

```
-PostHooks FormKeysRepair `
-ResultJson C:\Priority\tmp\agent-formprep\last.json
```

```
# Other switches
# -WhatIf          resolve + print park count, no UPDATE, no web, no CLI
# -SkipCli         web only after park
# -SkipWeb         CLI only after park (fails if CLI does not flip locks)
# -Force           ignore office-hours advisory (mutex still required)
# -Scope Named     default
# -Scope AllUnprepared FORBIDDEN unless -ChangeTicket <id> is also passed
```

Exit code 0 if and only if Ok=true. Exit 2 = environment/auth/mutex refuse (no park). Exit 3 = park taken but prep failed or restore short. Exit 4 = post-hook assertion failed (forms compiled but keys stripped).

5.2 Result JSON (authoritative contract)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "PrepareFormsResult",
  "required": ["ok", "runId", "environment", "startedAt", "endedAt", "durationSec",
    "parkedCount", "restoredCount", "prepared", "stillUnprepared",
    "errors", "dialogs", "executor"],
  "properties": {
    "ok": { "type": "boolean" },
    "runId": { "type": "string", "pattern": "^[0-9a-f-]{36}$" },
    "environment": { "const": "DEV" },
    "executor": { "enum": ["none", "cli", "web", "cli+web"] },
    "startedAt": { "type": "string", "format": "date-time" },
    "endedAt": { "type": "string", "format": "date-time" },
    "durationSec": { "type": "integer", "minimum": 0 },
    "parkedCount": { "type": "integer", "minimum": 0 },
    "restoredCount": { "type": "integer", "minimum": 0 },
    "restoreOk": { "type": "boolean" },
    "prepared": {
      "type": "array",
      "items": {
        "required": ["name", "execId", "lastPrepDateBefore", "lastPrepDateAfter", "upd"],
        "properties": {
          "name": { "type": "string" },
          "execId": { "type": "integer" },
          "lastPrepDateBefore": { "type": ["string", "number", "null"] },
          "lastPrepDateAfter": { "type": ["string", "number", "null"] },
          "upd": { "enum": ["N"] },
          "computer": { "type": "string" },
          "pid": { "type": "integer" }
        }
      }
    },
    "stillUnprepared": { "type": "array" },
    "errors": {
      "type": "array",
      "items": {
        "required": ["source", "text", "severity"],
        "properties": {
          "source": { "enum": ["modal", "errors-report", "emsg", "prep.err", "execpreblock", "hook"] },
          "formHint": { "type": "string" },
          "text": { "type": "string" },
          "severity": { "enum": ["Info", "Warning", "Blocker"] }
        }
      }
    },
    "dialogs": {
      "type": "array",
      "items": {
        "required": ["text", "action"],
        "properties": {
          "text": { "type": "string" },
          "action": { "enum": ["captured-left", "dismissed-safe", "blocked-run"] }
        }
      }
    },
    "auth": { "enum": ["ok", "expired", "unknown"] },
    "whatIf": { "type": "boolean" }
  }
}
```

Ok is true only when: every requested name is in prepared[], restoreOk is true, restoredCount==parkedCount, and no errors[].severity==Blocker names a target. UI completion is not a success signal.

6. Data catalog (implement against live DEV, then pin)

Priority SQL names vary by version and company. The build agent's first coding hour is not automation — it is a read-only reconnaissance query that dumps the real objects. Pin the result in config/dev.psd1. Do not guess column types.

6.1 Recon queries (read-only)

```
-- Run on 10.220.0.5\DEV, Windows integrated, database list: system and the company DB used by web (often company/base).
SELECT DB_NAME() AS db, SUSER_SNAME() AS login, HOST_NAME() AS host;

-- Find prep lock table
SELECT s.name AS sch, t.name AS tbl
FROM sys.tables t JOIN sys.schemas s ON s.schema_id=t.schema_id
WHERE t.name LIKE '%PREP%' OR t.name LIKE '%EXEC%'
ORDER BY t.name;

-- Candidate columns on EXECPREPLOCK / EXEC
SELECT c.name, ty.name, c.max_length
FROM sys.columns c
JOIN sys.types ty ON ty.user_type_id=c.user_type_id
WHERE c.object_id = OBJECT_ID('dbo.EXECPREPLOCK'); -- adjust once found

-- Resolve form names the way Priority does
-- Historical CE pattern: EXEC / T$EXEC with ENAME, EXEC (=numeric id)
SELECT TOP 20 *
FROM dbo.EXEC -- or T$EXEC / EXEC in company DB
WHERE ENAME LIKE 'ZCLA_PARTLONG%';
```

Pin after recon (example shape — replace after the SELECT works):

```
# config/dev.psd1
@{
    Environment      = 'DEV'
    WebBaseUrl       = 'https://prioritydev.clarkson-evans.co.uk'
    SqlInstance      = '10.220.0.5\DEV'
    SqlDatabase      = '<PIN>' # company DB that owns EXECPREPLOCK
    ExecTable        = '<PIN>' # e.g. dbo.EXEC or dbo.T$EXEC
    ExecNameCol      = 'ENAME'
    ExecIdCol        = 'EXEC'
    LockTable        = 'dbo.EXECPREPLOCK'
    LockCols         = @{
        ExecId='EXEC'; Upd='UPD'; LastPrep='LASTPREPDATE'
        Computer='COMPUTERNAME'; Pid='PID'; LockExpiry='LOCKEXPIRY'
    }
    ParkTable        = 'dbo.AGENT_FORMPREP_PARK' # created by sql/001
    PriorityUser      = 'Si' # WINRUN username only, no password
    Company          = 'base'
}
```

If EXECPREPLOCK does not exist under that name, search for UPD + LASTPREPDATE together. That pair is the source of truth cited by CE operators. Do not invent a substitute flag.

7. Park / restore protocol (highest-risk code)

Parking other unprepared forms by setting UPD='N' is Option A applied to everyone else. It is allowed only as a time-boxed isolation that is always reversed. A failed restore is a Sev-1 on DEV: ~1000 forms look prepared and are not.

7.1 Park table (source of truth — not a txt file)

```
-- sql/001_agent_formprep_park.sql
IF OBJECT_ID('dbo.AGENT_FORMPREP_PARK') IS NULL
CREATE TABLE dbo.AGENT_FORMPREP_PARK (
    run_id          UNIQUEIDENTIFIER NOT NULL,
    exec_id         INT              NOT NULL,
    ename           NVARCHAR(64)     NULL,
    prev_upd        CHAR(1)          NOT NULL,
    prev_lastprep   DATETIME         NULL,
    prev_computer   NVARCHAR(128)    NULL,
    prev_pid        INT              NULL,
    parked_at       DATETIME         NOT NULL DEFAULT GETDATE(),
    restored_at     DATETIME         NULL,
```

```

    restore_ok      CHAR(1)      NULL,
    PRIMARY KEY (run_id, exec_id)
);
CREATE INDEX IX_AGENT_FORMPREP_PARK_OPEN
ON dbo.AGENT_FORMPREP_PARK(restored_at) WHERE restored_at IS NULL;

```

A side-car text file C:\Priority\tmp\agent-formprep\{runId}\parked_execs.txt is allowed as a human-readable dump. It is not the restore source. Do not use C:\Users\medatech.si\Documents\wp1_parked_unprep_execs.txt in the new runner (legacy scripts may still read it; new code must not).

7.2 Park algorithm

- Acquire mutex Global\CE-DEV-FORMPREP (CreateMutex). If wait > 5s, exit 2 mutex_held. Do not park.
- Abort if any OPEN park rows exist (restored_at IS NULL) from another run_id. Print that run_id. Exit 2.
- Abort if any target or any UPD='Y' row has a non-stale lock (PID<>0 and LOCKEXPIRY in the future, or COMPUTERNAME set and host pingable). Exit 2 lock_held.
- Resolve names → exec ids. Abort if any name missing. Abort if Names.Count==0.
- Snapshot targets: current UPD, LASTPREPDATE, lock columns. Store in the result JSON as lastPrepDateBefore.
- Set targets: clear stale locks (COMPUTERNAME="", PID=0, LOCKEXPIRY=0), UPD='Y'. Do **not** zero LASTPREPDATE unless -ResetLastPrepDate (AT1 only).
- SELECT all other rows WHERE UPD='Y' AND exec_id NOT IN targets. INSERT them into AGENT_FORMPREP_PARK with prev_* values inside one transaction.
- Same transaction: UPDATE those rows SET UPD='N'. Commit.
- Assert COUNT(*) FROM lock WHERE UPD='Y' == target count. If not, restore immediately and exit 3 park_assert.
- Write the text dump. Log parkedCount.

7.3 Restore algorithm (always in finally)

- For every park row with this run_id AND restored_at IS NULL: UPDATE lock table back to prev_upd / prev_lastprep / prev_computer / prev_pid.
- Set restored_at=GETDATE(), restore_ok='Y' only if @@ROWCOUNT=1 per exec_id.
- restoredCount = rows with restore_ok='Y'. restoreOk = (restoredCount==parkedCount) AND (open rows for run_id == 0).
- If restoreOk is false: Ok=false, severity Blocker source=execpreplock, text='restore short: n/m', exit 3 even if targets compiled.
- Release mutex after restore, not before.

7.4 Crash recovery

Prepare-Forms -RepairOpenParks lists OPEN park rows and can restore them with -RepairOpenParks -RunId <guid>. A scheduled watchdog (optional Phase 2) alerts if any park row is older than 30 minutes.

8. End-to-end sequence

#	Step	Fail action
0	Assert-Environment: computer, SQL instance, web host, PriorityRoot, mutex	Exit 2, no park
1	Detect web session (storageState or computerUse profile). If login page, auth_expired	Exit 2, no park
2	Resolve Names → exec ids (sql/002)	Exit 2
3	WhatIf? print plan and stop	Exit 0 with whatIf=true
4	Park (section 7.2)	Restore anything written; exit 3
5	CLI probe unless -SkipCli: WINACTIV -P FORMPREP with NO form name, timeout CliTimeoutSeconds	On timeout, leave park in place, go to 6
6	SQL poll every 5s. If all targets UPD=N and LASTPREPDATE advanced → skip web	Else step 7
7	Web Form Prep unless -SkipWeb (section 9)	Capture dialogs; do not click dangerous buttons
8	Wait progress OR TimeoutMinutes. Poll SQL 5–10s after UI close	Continue to verify
9	Scrape errors (section 10)	Attach to result
10	Verify targets (section 11)	Ok=false if any still Y
11	Restore park (section 7.3) in finally	Exit 3 if short

#	Step	Fail action
12	Post-hooks (section 12)	Exit 4 if assertion fails
13	Write result JSON + log bundle, release mutex	—

9. Executors

9.1 CLI probe (after park only)

Use the existing WINRUN / WINACTIV pattern. Do not put a password on the command line in committed scripts. Read it from a Windows credential store or an already-authenticated session on DEV1. Example shape only:

```
# Invoke-CliFormPrep.ps1 - form name OMITTED on purpose
# After park, the unprepared set == targets, so "all unprepared" is the named set.
$winrun = 'C:\Priority\bin.95\winrun.exe'
$prep = 'C:\Priority\system\prep'
# Credential: Windows Credential Manager target 'CE/Priority/Si' - never commit the secret.
& $winrun '' $siUser $siPass $prep $company WINACTIV -P FORMPREP
# Do NOT append a form name here.
# Do NOT add -nbg until a human has proven background prep flips EXECPREPLOCK on this build.
```

Success for the CLI probe is SQL, not process exit code. Many CE runs exit 0 and do nothing. Wait up to CliTimeoutSeconds polling EXECPREPLOCK. If nothing moved, treat CLI as a no-op and continue to web. If CLI moves some but not all targets, still run web (executor=cli+web) unless the remaining targets have blocker errors.

9.2 Web Form Prep

Reuse the existing computerUse / Playwright skill that already walks: dashboard → My Shortcuts → Form Preparation → Prep Type Unprepared Forms (leave default) → OK → wait progress → Show Reports / Documents. Pin selectors in Invoke-WebFormPrep.ps1 as constants with a comment of the last date they worked.

- If the login page is shown at any point after park: stop clicking, restore, exit 3 auth_expired_after_park.
- Show Reports: OK is a safe dismiss (needed to open Errors Report).
- Cancel on Show Reports hides evidence — do not click Cancel.
- Priority Error modal: capture full text into dialogs[]. If text matches index/duplicate/trigger compile → action=blocked-run, do not click Ignore / Yes on rebuild. Abort remaining UI.
- Unknown modal: capture, action=blocked-run, abort.
- Screenshot every modal and the progress window into the run log dir.
- Popup blocker: if Errors Report tab never appears, do not fail solely for that; fall back to e*msg + prep.err + SQL.
- One Form Prep UI at a time. If progress from another session is visible, abort and restore.

9.3 Thick-client Form Generator (not MVP)

Phase 3 only. Open Form Generator on the named form, Prepare that form. Same verify + restore rules. Do not start this until AT1 passes on web or CLI.

10. Error capture matrix

Source	Where	Severity heuristic
Priority Error modal	UI text + screenshot	Blocker if names target/table/index; else Warning
Errors Report HTML	Report tab or exported HTML	Blocker if line contains target ENAME / TRIG; else Info
e*msg	C:\Priority\tmpl\e*msg (latest by LastWriteTime)	Info + 'Number of errors = N'; Warning if N>0
prep.err	C:\Priority\tmp\prep.err or equivalent if present	Info; noisy expanded SQL; keyword-filter targets
EXECPREPLOCK	SQL after UI/CLI done	Blocker if target still UPD='Y'
Hook	FORMKEYS / FORMJOINS assertion	Blocker if expected key missing after UPD='N'

Copy raw files into C:\Priority\tmp\agent-formprep\{runId}\capture\ so a later human can see what the agent saw. Redact nothing except credentials if a dialog ever echoes them.

11. Success criteria (strict)

For every requested form, all of the following must hold:

- EXECPREPLOCK.UPD = 'N'
- LASTPREPDATE is not null and is strictly greater than lastPrepDateBefore recorded in step 4 (or, only if -ResetLastPrepDate, greater than the run start timestamp)
- No Blocker error with formHint equal to that name
- restoreOk is true

Web progress finished, CLI exit 0, or LASTPREPDATE left at 0 after a reset, are all insufficient. A form that compiled with stripped FORMKEYS is Ok=false (exit 4) even if UPD='N'.

12. Post-hooks

Form Prep on CE has stripped FORMKEYS / expressions on known custom forms. Hooks run only after SQL says compiled. They are assertions plus restorations, not a substitute for prep.

```
# hooks/formkeys.psd1 – fill from a known-good dump, not from memory
@{
    ZCLA_PARTLONGDESC = @(
        @{ Col='PART'; KeyType='<PIN>'; Notes='example – replace from DEV dump' }
    )
    ZCLA_PARTLONGDHIST = @()
    ZCLA_PARTLONGDREV = @()
}
```

Apply-FormKeysRepair.ps1: for each expected key, SELECT from FORMKEYS (pin real table name in recon). If missing, re-insert the documented row, set that form UPD='Y' again, and either recurse Prepare-Forms once with -SkipPark (forbidden on first implementation) or return Blocker hook and leave the form for a human. MVP: assert-only. Do not invent INSERT statements until a human pastes a known-good dump into hooks/.

13. Auth, credentials, session

SQL: Windows integrated from the agent account on DEV1. No SQL password in repo or in JSON logs.

Priority web: Playwright storageState at C:\Priority\tmp\agent-formprep\si-web-state.json (ACL: the agent account only) or the existing computerUse profile. A human refreshes that file when AT7 fails. Detect auth by URL/title containing login / sign in / Logon, or by absence of the dashboard shortcuts tile within 20s.

WINRUN Si password: Windows Credential Manager target CE/Priority/Si, or an env var injected by the host — never a file in git, never this PDF, never the result JSON.

14. Logging and artefacts

```
C:\Priority\tmp\agent-formprep\
{runId}\
  result.json
  parked_execs.txt          (dump only)
  targets.csv
  sql-before.json
  sql-after.json
  cli-stdout.txt
  web-trace.ndjson
  capture\
    modal-01.png
    progress.png
    errors-report.html
    emsg.txt
    prep.err
  last.json -> latest result
  si-web-state.json         (gitignored)
```

Gitignore the tmp tree. Commit schemas, sql, hooks dumps that contain no secrets, and this PDF.

15. Implementation phases and work packages

ID	Package	Done when
WP0	Recon + config/dev.psd1 pinned	SELECT on lock table works; three ZCLA names resolve
WP1	Mutex + park table + WhatIf + RepairOpenParks	AT4 dry: park 5 dummy rows, kill process, repair restores 5/5
WP2	Verify + error scrape without running prep	Can dump EXECPREPLOCK + latest e*msg for named forms

ID	Package	Done when
WP3	CLI probe on a parked 1-form set	Either LASTPREPDATE moves or probe returns no-op with evidence
WP4	Web Form Prep automation + dialog policy	AT1 green on 3 forced-Y forms; screenshots in run dir
WP5	JSON schema + Prepare-Forms.ps1 public surface	Eshbel can call it; exit codes match section 5
WP6	Post-hook assertions from a human dump	AT1 fails if FORMKEYS missing
WP7	Docs in form-prep/README.md + watchdog query	Open parks older than 30 min are visible

Do not start WP4 until WP1 AT4 is green. Do not add AllUnprepared, Form Generator, or REST discovery before WP5.

16. Acceptance tests

ID	Setup	Expect
AT1	Three already-prepared forms forced UPD='Y' (do not zero LASTPREPDATE unless flag). Park + prep.	All three UPD=N, LASTPREPDATE advanced, Ok=true, restoredCount==parkedCount, FORMKEYS still present
AT2	Throwaway form ZCLA_AGENT_PREP_AT2 with broken trigger text. Must exist only on DEV.	Ok=false, Errors[] formHint=that name, park restored, throwaway trigger restored after test
AT3	Force an index-rebuild / IGNORE_DUP_KEY dialog if reproducible; else replay a saved modal screenshot fixture.	dialogs[].action=blocked-run; runner did not click Ignore; Ok=false
AT4	Park a known set, kill the PowerShell process mid-web or mid-sleep.	RepairOpenParks restores exact prev_upd counts; no net change to non-target UPD flags
AT5	Optional control: no park, full queue web. Document how many high ids stay Y.	Not required to pass MVP; write the number into README
AT6	Point config at a fake SqlInstance or WebBaseUrl.	Exit 2, zero rows inserted into park table
AT7	Corrupt or delete si-web-state so the login page shows.	Exit 2 auth_expired, zero park rows
AT8	Second Prepare-Forms while the first holds the mutex.	Exit 2 mutex_held, no nested park

17. Risks and mitigations

Risk	Mitigation
Failed restore marks ~1000 forms prepared	SQL park table, finally, restoreOk required for Ok, watchdog, RepairOpenParks
Si web session dies overnight	AT7; do not park if login page; human refresh of storageState
Popup blocker hides Errors Report	e*msg + prep.err + SQL still required
Index dialog auto-OK changes keys	blocked-run policy; no Ignore click
Form Prep strips FORMKEYS	assert-only hooks in MVP; human dump in hooks/
Two agents park each other	Global mutex + reject OPEN park rows
CLI false exit 0	SQL is the only success signal
Someone runs this on PRI	Assert-Environment constants; -Environment DEV required
Office-hours collision with humans	Mutex; optional -Force; log computer/pid
Password leaked into logs	Redact WINRUN command line in cli-stdout.txt

18. Operator cheat-sheet

```
# First-time on DEV1 (human)
# 1. Confirm hostname and SQL window
# 2. Run sql/001 against the pinned company DB
# 3. Log into Priority web once; save Playwright storageState
# 4. Store Si WINRUN secret in Credential Manager CE/Priority/Si
# 5. WhatIf
powershell -File src\Prepare-Forms.ps1 -Names ZCLA_PARTLONGDESC -Environment DEV -WhatIf

# Real one-form run
powershell -File src\Prepare-Forms.ps1 -Names ZCLA_PARTLONGDESC -Environment DEV -TimeoutMinutes 15

# After a crash
powershell -File src\Prepare-Forms.ps1 -RepairOpenParks -Environment DEV
```



```
# or -RepairOpenParks -RunId <guid>

# Inspect
SELECT * FROM dbo.AGENT_FORMPREP_PARK WHERE restored_at IS NULL;
SELECT ENAME, UPD, LASTPREPDATE, COMPUTERNAME, PID
FROM dbo.EXECPREPLOCK L JOIN dbo.EXEC E ON E.EXEC = L.EXEC
WHERE E.ENAME LIKE 'ZCLA_PARTLONG%';
```

19. What the build agent must not do

- Do not UPDATE UPD='N' on targets and return Ok.
- Do not start web prep before park assert (COUNT UPD=Y == target count).
- Do not restore from a text file if the SQL park table has rows.
- Do not click through index/duplicate dialogs.
- Do not commit secrets, storageState, or result JSON that contains a password.
- Do not implement AllUnprepared in the same PR as WP1–WP5.
- Do not “fix” AT4 by skipping park.
- Do not run Form Prep twice concurrently to go faster.
- Do not change live/PRI connection strings “just to test.”

20. Definition of done for MVP

A Windows agent on CE-PRIORITY-DEV1 can take ZCLA_PARTLONGDESC, ZCLA_PARTLONGDHIST, ZCLA_PARTLONGDREV, return result.json with Ok=true, show LASTPREPDATE advanced on all three, restore every parked row, and leave screenshots plus e*msg in the run directory. AT1, AT4, AT6, AT7, AT8 are green. AT2/AT3 are green or explicitly waived in README with a reason. No password is in git. This PDF sits at form-prep/ in the repo.

Document status: v1.0 for implementation. Update config/dev.psd1 when recon pins real table names; bump this plan to v1.1 with those names rather than editing history in place.